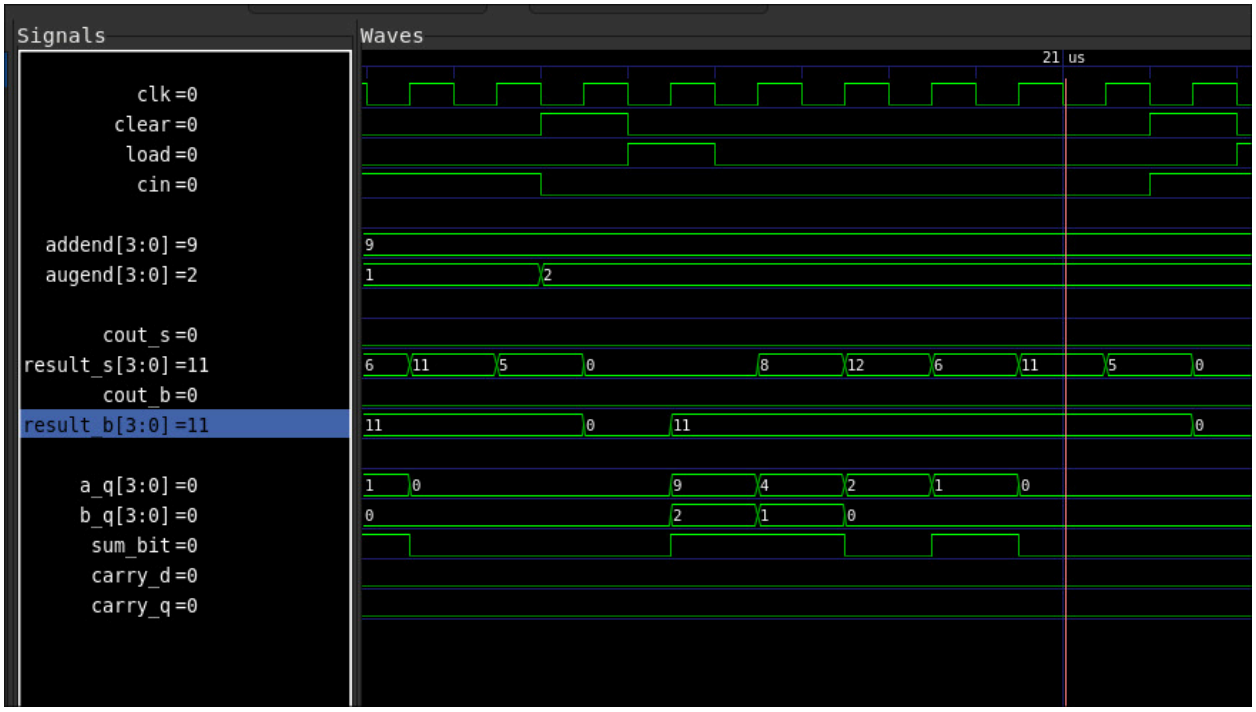
Group 10: Trevor Nichols, Pauls Olins, Cristian Sosa
ECSE 318 - Lab 1

This is a resubmission of lab 1 with the added explanation about critical path in part B of problem 2

Problem 1

- Waves:



- Log:

```
None
# PASS: 5 + 1 + 0 = 6 (cout = 0)  structural and behavioral agree
# PASS: 5 + 0 + 1 = 6 (cout = 0)  structural and behavioral agree
# PASS: 0 + 0 + 0 = 0 (cout = 0)  structural and behavioral agree
# PASS: 0 + 0 + 1 = 1 (cout = 0)  structural and behavioral agree
# PASS: 15 + 15 + 0 = 14 (cout = 1)  structural and behavioral agree
# PASS: 15 + 15 + 1 = 15 (cout = 1)  structural and behavioral agree
# PASS: 10 + 7 + 1 = 2 (cout = 1)  structural and behavioral agree
# ------------------------------------------------
# vectors tested : 519
# correctness    : PASS - both models match addend+augend+cin for every
vector
```

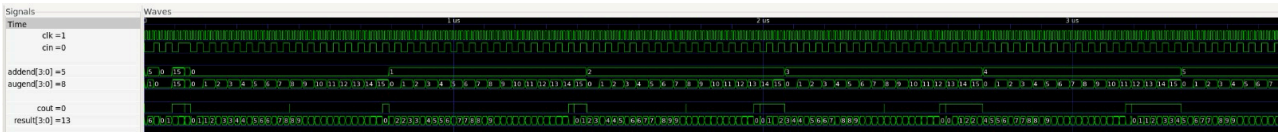
```
# equivalence    : PASS - structural and behavioral outputs identical for
all 519 vectors
# ALL TESTS PASSED
```

c) Testbench simulated all possible inputs for both the structural and the behavioral designs, all outputs matched, so the two models are equivalent.

Problem 2

For parts A and C:

- Waves:



- Log:

```
None
# PASS: 5 + 1 + 0 = 6 (cout = 0)
# PASS: 5 + 0 + 1 = 6 (cout = 0)
# PASS: 0 + 0 + 0 = 0 (cout = 0)
# PASS: 0 + 0 + 1 = 1 (cout = 0)
# PASS: 15 + 15 + 0 = 14 (cout = 1)
# PASS: 15 + 15 + 1 = 15 (cout = 1)
# PASS: 10 + 7 + 1 = 2 (cout = 1)
# ------------------------------------------------
# vectors tested : 519
# correctness    : PASS - both models match addend+augend+cin for every
vector
# ALL TESTS PASSED
```

Part B:

For longest delay from C0 to sum MSB, the start is cin and the end point is result[3]. result[3]=p3 XOR carry [2]. C0 can only reach result[3] through carry[2].

To force C0 to travel all the way to output:

- Every AND on the path needs the other input to be 1, so p0=p1=p2=1;
- Every OR on the path needs the other input to be 0;

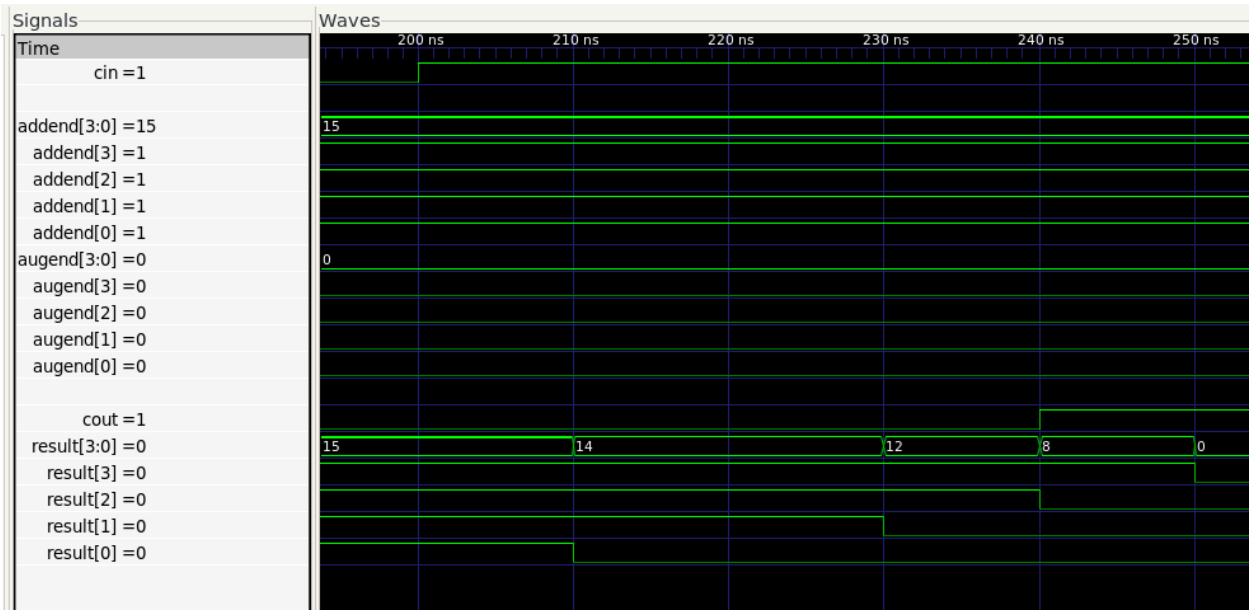
So requirement is A XOR B = x111. An input to achieve this could be A=1111, B=0000, C0: changes from 0 to 1.
Our implementation uses 2-input gates (and2, or2) in trees for the carry generate block logic, so the path is like this:

```
None
AND: Cin & p0
AND: (Cin & p0) & (p1 & p2)
OR: term2 | term3
OR: (t0 | t1) | (t2 | t3)
XOR: p3 ^ carry[2]
```

With 5 gates and gate level being 10, the longest path is 50.
Simulation log for tb testing this (part 1 is with the input shown above and part2 tests all inputs changing C0 to confirm the longest delay):

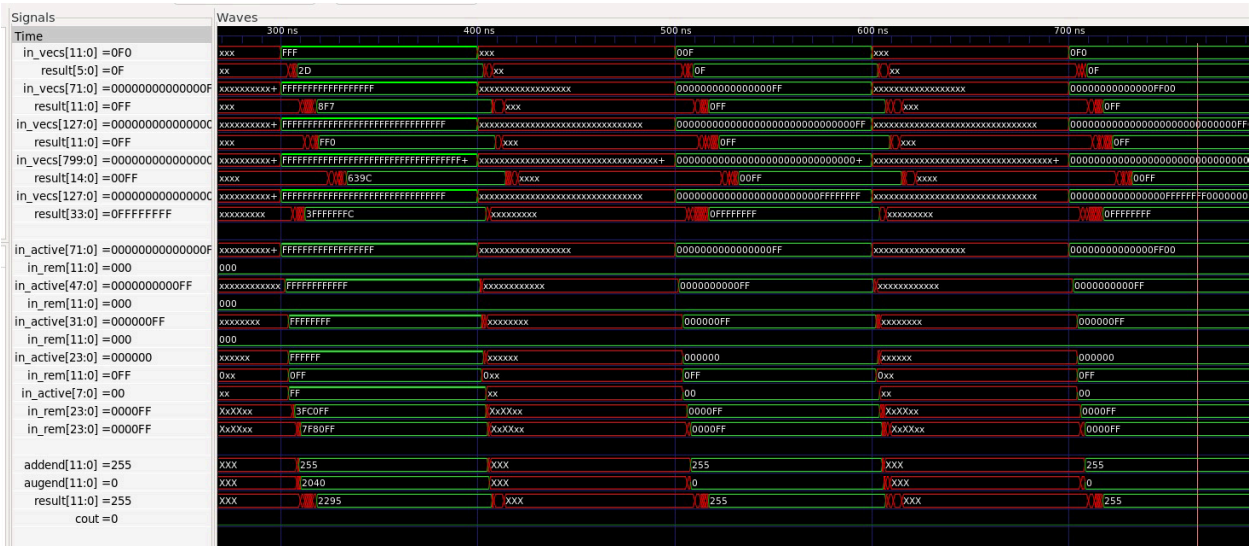
```
None
# Part 1: A=1111 B=0000, cin 0 -> 1 (p = 1111, g = 0000)
#   t=10  carry[2] cin-AND first level (cin&p0) = 1
#   t=10  result = 1110
#   t=20  carry[2] terms = 1000
#   t=20  carry  = 0001
#   t=30  result = 1100
#   t=30  carry[2] OR first level (term2|term3) = 1
#   t=30  carry  = 0011
#   t=40  cout   = 1
#   t=40  result = 1000
#   t=40  carry  = 1111
#   t=50  result = 0000
#   => result[3] settled at 50, cout at 40 (after cin changed)
# Part 2: slowest C0 -> result[3] over all inputs = 50
#         first found at A=0000 B=0111, cin 0 -> 1
```

Screenshot from waveform showing the gate propagation times:



Problem 3

- Waves:



- Log:

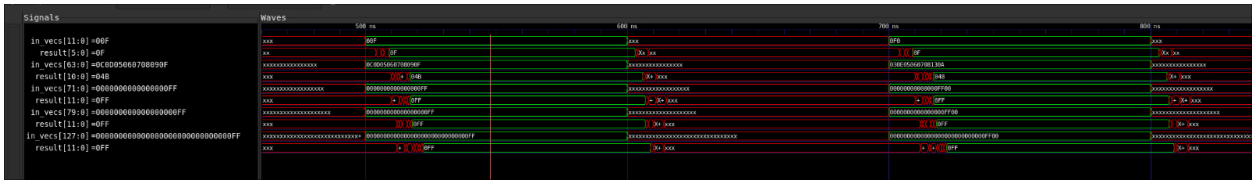
```
None
# PASS N=1 M=1 (303 vectors, W=1)
# DELAY N=1 M=1: min=20 avg=20 max=20 spread=0 (303 changed, 0 no-change)
# PASS N=8 M=1 (303 vectors, W=8)
# DELAY N=8 M=1: min=20 avg=20 max=20 spread=0 (303 changed, 0 no-change)
# PASS N=1 M=2 (304 vectors, W=2)
# DELAY N=1 M=2: min=30 avg=30 max=30 spread=0 (304 changed, 0 no-change)
```

```
# PASS N=8 M=2 (304 vectors, W=9)
# DELAY N=8 M=2: min=60 avg=83 max=90 spread=30 (304 changed, 0 no-change)
# PASS N=1 M=3 (305 vectors, W=3)
# DELAY N=1 M=3: min=40 avg=43 max=50 spread=10 (305 changed, 0 no-change)
# PASS N=4 M=3 (305 vectors, W=6)
# DELAY N=4 M=3: min=80 avg=91 max=100 spread=20 (305 changed, 0
no-change)
# PASS N=8 M=3 (305 vectors, W=10)
# DELAY N=8 M=3: min=90 avg=109 max=120 spread=30 (305 changed, 0
no-change)
# PASS N=4 M=4 (306 vectors, W=6)
# DELAY N=4 M=4: min=90 avg=109 max=120 spread=30 (306 changed, 0
no-change)
# PASS N=8 M=4 (306 vectors, W=10)
# DELAY N=8 M=4: min=100 avg=127 max=140 spread=40 (306 changed, 0
no-change)
# PASS N=32 M=4 (306 vectors, W=34)
# DELAY N=32 M=4: min=120 avg=164 max=180 spread=60 (306 changed, 0
no-change)
# PASS N=3 M=5 (307 vectors, W=6)
# DELAY N=3 M=5: min=100 avg=123 max=140 spread=40 (307 changed, 0
no-change)
# PASS N=8 M=5 (307 vectors, W=11)
# DELAY N=8 M=5: min=130 avg=148 max=160 spread=30 (307 changed, 0
no-change)
# PASS N=8 M=6 (308 vectors, W=11)
# DELAY N=8 M=6: min=120 avg=151 max=160 spread=40 (308 changed, 0
no-change)
# PASS N=2 M=7 (309 vectors, W=5)
# DELAY N=2 M=7: min=120 avg=132 max=150 spread=30 (309 changed, 0
no-change)
# PASS N=8 M=7 (309 vectors, W=11)
# DELAY N=8 M=7: min=140 avg=166 max=180 spread=40 (309 changed, 0
no-change)
# PASS N=8 M=8 (310 vectors, W=11)
# DELAY N=8 M=8: min=140 avg=181 max=200 spread=60 (310 changed, 0
no-change)
# PASS N=4 M=9 (311 vectors, W=8)
# DELAY N=4 M=9: min=140 avg=160 max=180 spread=40 (311 changed, 0
no-change)
```

```
# PASS N=8 M=9 (311 vectors, W=12)
# DELAY N=8 M=9: min=160 avg=181 max=200 spread=40 (311 changed, 0
no-change)
# PASS N=8 M=15 (317 vectors, W=12)
# DELAY N=8 M=15: min=190 avg=228 max=250 spread=60 (317 changed, 0
no-change)
# PASS N=8 M=16 (318 vectors, W=12)
# DELAY N=8 M=16: min=190 avg=235 max=260 spread=70 (318 changed, 0
no-change)
# PASS N=8 M=32 (334 vectors, W=13)
# DELAY N=8 M=32: min=230 avg=283 max=300 spread=70 (334 changed, 0
no-change)
# PASS N=16 M=32 (334 vectors, W=21)
# DELAY N=16 M=32: min=240 avg=301 max=320 spread=80 (334 changed, 0
no-change)
# PASS N=8 M=60 (362 vectors, W=14)
# DELAY N=8 M=60: min=290 avg=342 max=360 spread=70 (362 changed, 0
no-change)
# PASS N=4 M=64 (366 vectors, W=10)
# DELAY N=4 M=64: min=270 avg=317 max=340 spread=70 (366 changed, 0
no-change)
# PASS N=8 M=100 (402 vectors, W=15)
# DELAY N=8 M=100: min=320 avg=352 max=370 spread=50 (402 changed, 0
no-change)
```

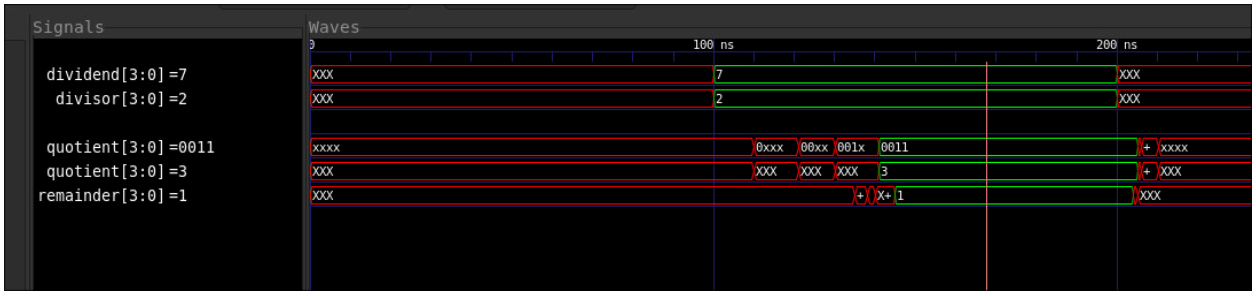
a) 10 = 7 = 5 = 4 = 3 = 2
Each CSA has 3 inputs and 2 outputs, so the total number of CSA gates is 2 less than the number of inputs (for 10 input numbers we have 8 gates). With 10 inputs stages:
Stage 1 has 3 gates, 2 * 3 outputs, 6 + 1 = 7;
Stage 2 has 2 gates, 2 * 2 outputs, 4 + 1 = 5;
Stage 3 has 1 gate, 2 * 1 outputs, 2 + 2 = 4;
Stage 4 has 1 gate, 2 * 1 outputs, 2 + 1 = 3;
Stage 5 has 1 gate, 2 * 1 outputs, 2 + 0 = 2;
+ adder.
So for 10 inputs need 5 stages, 8 CSAs.

b) Simulation for sequences (12,13,5,6,7,8,9,15) and (3,14,5,6,7,8,19,10).

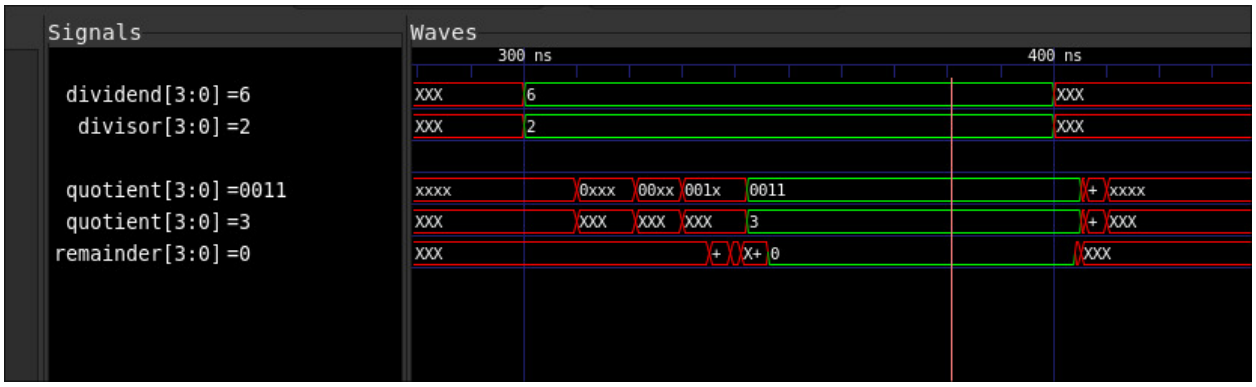


Problem 4

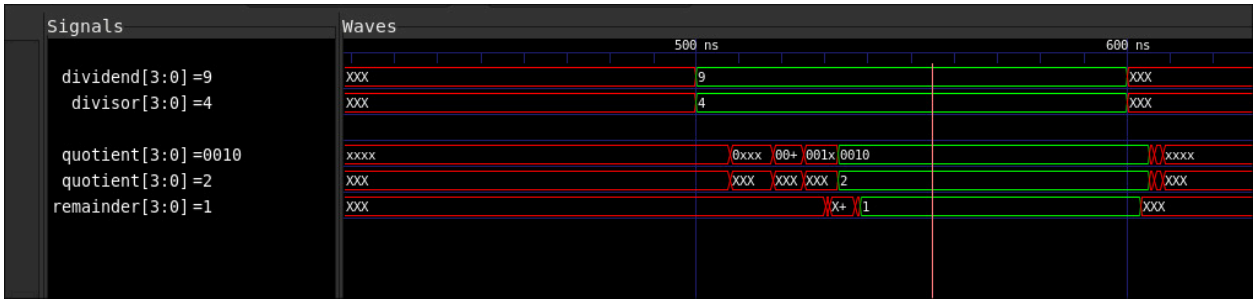
- Waves:
    - 7/2



    - 6/2



    - 9/4



- Log:

```
None
# PASS: 7 / 2 = 3 remainder 1
```

```
# PASS: 6 / 2 = 3 remainder 0
# PASS: 9 / 4 = 2 remainder 1
# PASS: 0 / 1 = 0 remainder 0
# PASS: 15 / 15 = 1 remainder 0
# PASS: 15 / 13 = 1 remainder 2
# PASS: 15 / 1 = 15 remainder 0
# ------------------------------------------------
# vectors tested : 247
# correctness    : PASS - q and r match divmod for every vector
# ALL TESTS PASSED
# ------------------------------------------------
# DELAY N=4: min=22 avg=34 max=57 spread=35 (247 changed, 0 no-change)
```

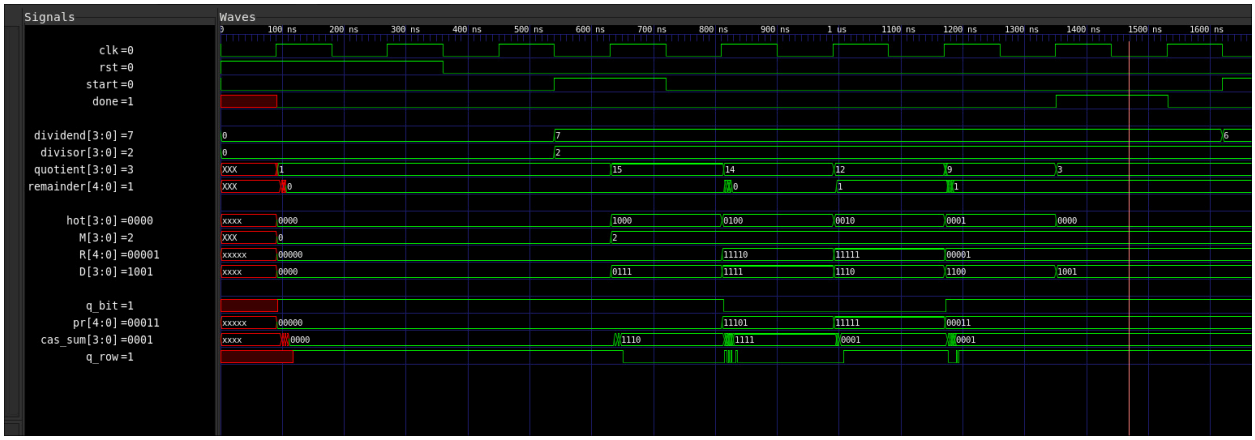
- Gate analysis:

```
None
divider_array  N=4
  gates      : and=50 or=23 xor=64 not=1
  total      : 138 gates (bufs excluded)
  max depth  : 58 gates (longest path)
```

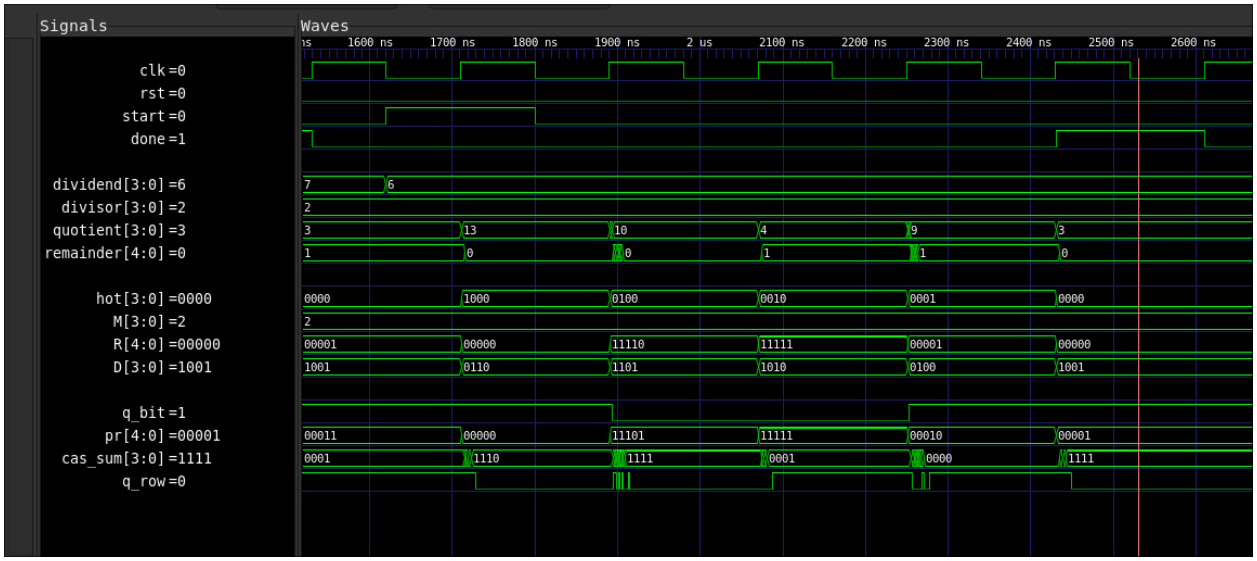
c) Time
In simulation the average time was 34ns, max was 57ns to complete operation. But with gate analysis, the max depth is 58 gates, so with 1ns per gate, the max time to complete an operation is 58ns.

Problem 5

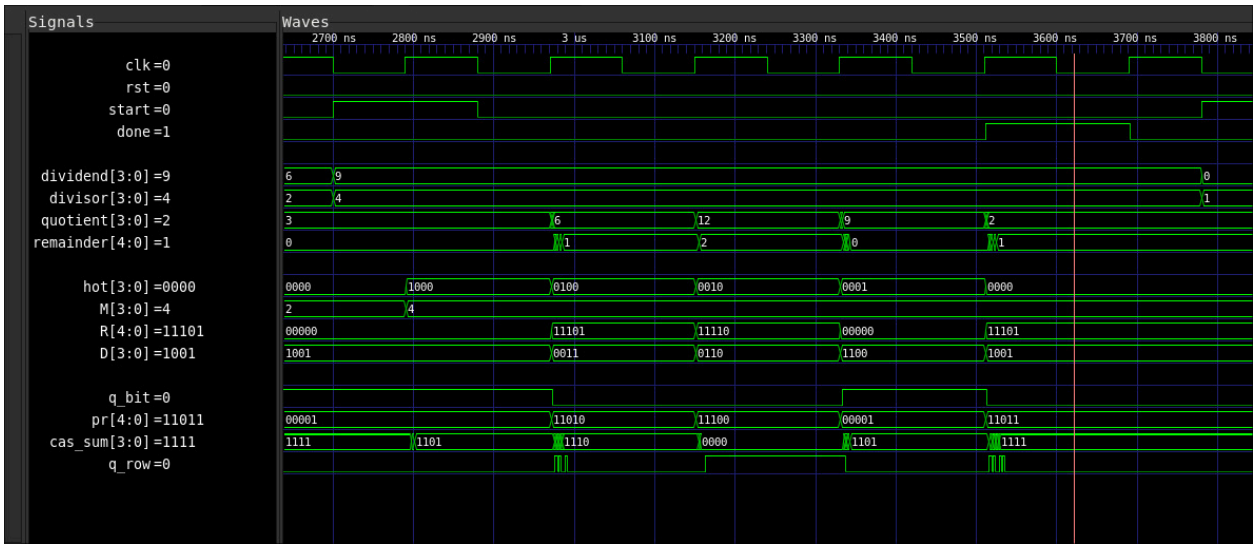
- Waves:
    - 7/2


Signals
Waves
clk =0
rst =0
start =0
done =1
dividend[3:0] =7
divisor[3:0] =2
quotient[3:0] =3
remainder[4:0] =1
hot[3:0] =0000
M[3:0] =2
R[4:0] =00001
D[3:0] =1001
q_bit =1
pr[4:0] =00011
cas_sum[3:0] =0001
q_row =1

- 6/2


Signals
Waves
clk =0
rst =0
start =0
done =1
dividend[3:0] =6
divisor[3:0] =2
quotient[3:0] =3
remainder[4:0] =0
hot[3:0] =0000
M[3:0] =2
R[4:0] =00000
D[3:0] =1001
q_bit =1
pr[4:0] =00001
cas_sum[3:0] =1111
q_row =0

- 9/4


Signals
Waves
clk =0
rst =0
start =0
done =1
dividend[3:0] =9
divisor[3:0] =4
quotient[3:0] =2
remainder[4:0] =1
hot[3:0] =0000
M[3:0] =4
R[4:0] =11101
D[3:0] =1001
q_bit =0
pr[4:0] =11011
cas_sum[3:0] =1111
q_row =0

- Log:

```
None
# PASS N=4: 7/2  quotient = 3  remainder = 1
# PASS N=4: 6/2  quotient = 3  remainder = 0
# PASS N=4: 9/4  quotient = 2  remainder = 1
# PASS N=4: 0/1  quotient = 0  remainder = 0
# PASS N=4: 15/15  quotient = 1  remainder = 0
# PASS N=4: 15/13  quotient = 1  remainder = 2
# PASS N=4: 15/1  quotient = 15  remainder = 0
# PASS N=2 (316 vectors)
# DELAY N=2: min=1 avg=14 max=29 spread=28 (1264 active cycles)
# PASS N=3 (360 vectors)
# DELAY N=3: min=1 avg=17 max=33 spread=32 (1800 active cycles)
# PASS N=8 (307 vectors)
# DELAY N=8: min=1 avg=23 max=53 spread=52 (3070 active cycles)
# PASS N=4 (547 vectors)
# DELAY N=4: min=1 avg=18 max=37 spread=36 (3282 active cycles)
# PASS N=16 (307 vectors)
# DELAY N=16: min=1 avg=30 max=85 spread=84 (5526 active cycles)
# PASS N=5 (1299 vectors)
# DELAY N=5: min=1 avg=20 max=41 spread=40 (9093 active cycles)
```

- Gate analysis:

```
None
divider  N=4
  gates      : and=101 or=50 xor=24 not=10
  registers  : 18 flip-flops in 5 cells (clear muxes: 18 bits in 5 cells)
  total      : 185 gates (bufs and registers excluded)
  max depth  : 19 gates (longest register-to-register path)
```

c) Time

For our simulation we used gate delay time 2 and flip-flop delay time 1.

For 4 bit simulation:

```
# DELAY N=4: min=1 avg=18 max=37 spread=36 (3282 active cycles)
```

Average time was 18ns and max delay time was 37ns.

If assuming the gate time delay of 1ns and 1/2ns for flip flop setup and hold delay, the time to complete operation is 18/2=9ns average and 37/2=18.5ns max.